```
# Name : Prathamesh Lavange
# div : B
# Roll No :31
# PRN : 72332154L
```

```
import pandas as pd
import numpy as np

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

print("Libraries imported successfully!")
```

```
Libraries imported successfully!
```

```
df = pd.read_csv(
    "https://raw.githubusercontent.com/dsaif4/Telecom-Churn-Dataset/main/churn-bigml-80.csv"
)

display(df.head())
print("Shape:", df.shape)
```

	State	Account length	Area code	International plan	Voice mail plan	Number vmail messages	Total day minutes	Total day calls	Total day charge	Total eve minutes	Total eve calls	Total eve charge	Total night minutes	Total night calls	Total night charge	m
0	KS	128	415	No	Yes	25	265.1	110	45.07	197.4	99	16.78	244.7	91	11.01	
1	OH	107	415	No	Yes	26	161.6	123	27.47	195.5	103	16.62	254.4	103	11.45	
2	NJ	137	415	No	No	0	243.4	114	41.38	121.2	110	10.30	162.6	104	7.32	
3	OH	84	408	Yes	No	0	299.4	71	50.90	61.9	88	5.26	196.9	89	8.86	
4	OK	75	415	Yes	No	0	166.7	113	28.34	148.3	122	12.61	186.9	121	8.41	

```
Shape: (2666, 20)
```

```
# Step 4: Dataset Information

print("Dataset Shape:")
print(df.shape)

print("\nColumn Names:")
print(df.columns.tolist())

print("\nDataset Information:")
df.info()

print("\nStatistical Summary:")
display(df.describe(include="all"))

print("\nChurn Distribution:")
print(df["Churn"].value_counts())
```

```
Dataset Shape:
(2666, 20)

Column Names:
['State', 'Account length', 'Area code', 'International plan', 'Voice mail plan', 'Number vmail messages', 'Total day minutes',

Dataset Information:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 2666 entries, 0 to 2665
Data columns (total 20 columns):
 #   Column                  Non-Null Count  Dtype
---  ------                  --------------  -----
 0   State                   2666 non-null   object
 1   Account length          2666 non-null   int64
 2   Area code               2666 non-null   int64
 3   International plan      2666 non-null   object
 4   Voice mail plan         2666 non-null   object
 5   Number vmail messages   2666 non-null   int64
 6   Total day minutes       2666 non-null   float64
 7   Total day calls         2666 non-null   int64
 8   Total day charge        2666 non-null   float64
 9   Total eve minutes       2666 non-null   float64
 10  Total eve calls         2666 non-null   int64
 11  Total eve charge        2666 non-null   float64
 12  Total night minutes     2666 non-null   float64
 13  Total night calls       2666 non-null   int64
 14  Total night charge      2666 non-null   float64
 15  Total intl minutes      2666 non-null   float64
 16  Total intl calls        2666 non-null   int64
 17  Total intl charge       2666 non-null   float64
 18  Customer service calls  2666 non-null   int64
 19  Churn                   2666 non-null   bool
dtypes: bool(1), float64(8), int64(8), object(3)
memory usage: 398.5+ KB

Statistical Summary:
```

	State	Account length	Area code	International plan	Voice mail plan	Number vmail messages	Total day minutes	Total day calls	Total day charge	Total eve minutes	Tot
count	2666	2666.000000	2666.000000	2666	2666	2666.000000	2666.00000	2666.000000	2666.000000	2666.000000	2666.
unique	51	NaN	NaN	2	2	NaN	NaN	NaN	NaN	NaN	
top	WV	NaN	NaN	No	No	NaN	NaN	NaN	NaN	NaN	
freq	88	NaN	NaN	2396	1933	NaN	NaN	NaN	NaN	NaN	
mean	NaN	100.620405	437.438860	NaN	NaN	8.021755	179.48162	100.310203	30.512404	200.386159	100.
std	NaN	39.563974	42.521018	NaN	NaN	13.612277	54.21035	19.988162	9.215733	50.951515	20.
min	NaN	1.000000	408.000000	NaN	NaN	0.000000	0.00000	0.000000	0.000000	0.000000	0.
25%	NaN	73.000000	408.000000	NaN	NaN	0.000000	143.40000	87.000000	24.380000	165.300000	87.
50%	NaN	100.000000	415.000000	NaN	NaN	0.000000	179.95000	101.000000	30.590000	200.900000	100.
75%	NaN	127.000000	510.000000	NaN	NaN	19.000000	215.90000	114.000000	36.700000	235.100000	114.
max	NaN	243.000000	510.000000	NaN	NaN	50.000000	350.80000	160.000000	59.640000	363.700000	170.

```
Churn Distribution:
Churn
False    2278
True      388
Name: count, dtype: int64
```

```
print("Missing Values in Each Column:")
print(df.isnull().sum())
```

```
Missing Values in Each Column:
State                     0
Account length            0
Area code                 0
International plan        0
Voice mail plan           0
Number vmail messages     0
Total day minutes         0
Total day calls           0
Total day charge          0
Total eve minutes         0
Total eve calls           0
```

```
Total eve charge          0
Total night minutes       0
Total night calls         0
Total night charge        0
Total intl minutes        0
Total intl calls          0
Total intl charge         0
Customer service calls    0
Churn                     0
dtype: int64
```

```
# Step 6: Handle Missing Values

# Numerical columns मधील missing values median ने भरणे
num_cols = df.select_dtypes(include=np.number).columns

for col in num_cols:
    df[col] = df[col].fillna(df[col].median())

# Categorical columns मधील missing values mode ने भरणे
cat_cols = df.select_dtypes(
    include=["object", "category", "string"]
).columns

for col in cat_cols:
    mode_values = df[col].mode(dropna=True)

    if not mode_values.empty:
        df[col] = df[col].fillna(mode_values.iloc[0])

print("Missing values handled successfully!")

print("\nMissing values remaining:")
print(df.isnull().sum())
```

```
Missing values handled successfully!

Missing values remaining:
State                     0
Account length            0
Area code                 0
International plan        0
Voice mail plan           0
Number vmail messages     0
Total day minutes         0
Total day calls           0
Total day charge          0
Total eve minutes         0
Total eve calls           0
Total eve charge          0
Total night minutes       0
Total night calls         0
Total night charge        0
Total intl minutes        0
Total intl calls          0
Total intl charge         0
Customer service calls    0
Churn                     0
dtype: int64
```

```
print("Churn column values:")
print(df["Churn"].value_counts())

print("\nChurn data type:")
print(df["Churn"].dtype)
```

```
Churn column values:
Churn
False    2278
True      388
Name: count, dtype: int64

Churn data type:
bool
```

```
# Step 8: Detect Outliers using IQR

num_cols = df.select_dtypes(include=np.number).columns
```

```
for col in num_cols:
    Q1 = df[col].quantile(0.25)
    Q3 = df[col].quantile(0.75)

    IQR = Q3 - Q1

    lower_limit = Q1 - 1.5 * IQR
    upper_limit = Q3 + 1.5 * IQR

    outliers = df[
        (df[col] < lower_limit) |
        (df[col] > upper_limit)
    ]

    print(col, ":", len(outliers), "outliers")
```

```
Account length : 12 outliers
Area code : 0 outliers
Number vmail messages : 2 outliers
Total day minutes : 21 outliers
Total day calls : 18 outliers
Total day charge : 21 outliers
Total eve minutes : 17 outliers
Total eve calls : 15 outliers
Total eve charge : 17 outliers
Total night minutes : 22 outliers
Total night calls : 19 outliers
Total night charge : 22 outliers
Total intl minutes : 37 outliers
Total intl calls : 66 outliers
Total intl charge : 40 outliers
Customer service calls : 210 outliers
```

```
# Step 9: Handle Outliers using IQR

for col in num_cols:
    Q1 = df[col].quantile(0.25)
    Q3 = df[col].quantile(0.75)

    IQR = Q3 - Q1

    lower_limit = Q1 - 1.5 * IQR
    upper_limit = Q3 + 1.5 * IQR

    df[col] = df[col].clip(
        lower=lower_limit,
        upper=upper_limit
    )

print("Outliers handled successfully!")
display(df.head())
```

```
Outliers handled successfully!
```

	State	Account length	Area code	International plan	Voice mail plan	Number vmail messages	Total day minutes	Total day calls	Total day charge	Total eve minutes	Total eve calls	Total eve charge	Total night minutes	Total night calls	Total night charge	m
0	KS	128	415	No	Yes	25.0	265.1	110.0	45.07	197.4	99.0	16.78	244.7	91	11.01	
1	OH	107	415	No	Yes	26.0	161.6	123.0	27.47	195.5	103.0	16.62	254.4	103	11.45	
2	NJ	137	415	No	No	0.0	243.4	114.0	41.38	121.2	110.0	10.30	162.6	104	7.32	
3	OH	84	408	Yes	No	0.0	299.4	71.0	50.90	61.9	88.0	5.26	196.9	89	8.86	
4	OK	75	415	Yes	No	0.0	166.7	113.0	28.34	148.3	122.0	12.61	186.9	121	8.41	

```
# Step 10: Feature Engineering

df["Total minutes"] = (
    df["Total day minutes"]
    + df["Total eve minutes"]
    + df["Total night minutes"]
    + df["Total intl minutes"]
)

df["Total calls"] = (
```

```
    df["Total day calls"]
    + df["Total eve calls"]
    + df["Total night calls"]
    + df["Total intl calls"]
)

df["High service calls"] = (
    df["Customer service calls"] >= 4
).astype(int)

print("New features created successfully!")

display(df[[
    "Total minutes",
    "Total calls",
    "High service calls"
]].head())
```

```
New features created successfully!
```

	Total minutes	Total calls	High service calls
0	717.2	303.0	0
1	625.2	332.0	0
2	539.4	333.0	0
3	564.8	255.0	0
4	512.0	359.0	0

```
# Step 11: Separate Features and Target

X = df.drop("Churn", axis=1)
y = df["Churn"]

# Text columns चे numeric columns मध्ये रूपांतर
X = pd.get_dummies(X, drop_first=True, dtype=int)

print("Features (X) shape:", X.shape)
print("Target (y) shape:", y.shape)

display(X.head())
```

```
Features (X) shape: (2666, 71)
Target (y) shape: (2666,)
```

	Account length	Area code	Number vmail messages	Total day minutes	Total day calls	Total day charge	Total eve minutes	Total eve calls	Total eve charge	Total night minutes	...	State_TX	State_UT	State_VA	State_VT
0	128	415	25.0	265.1	110.0	45.07	197.4	99.0	16.78	244.7	...	0	0	0	0
1	107	415	26.0	161.6	123.0	27.47	195.5	103.0	16.62	254.4	...	0	0	0	0
2	137	415	0.0	243.4	114.0	41.38	121.2	110.0	10.30	162.6	...	0	0	0	0
3	84	408	0.0	299.4	71.0	50.90	61.9	88.0	5.26	196.9	...	0	0	0	0
4	75	415	0.0	166.7	113.0	28.34	148.3	122.0	12.61	186.9	...	0	0	0	0

5 rows × 71 columns

```
# Step 12: Train-Test Split

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42,
    stratify=y
)

print("Training Features:", X_train.shape)
print("Testing Features:", X_test.shape)

print("Training Target:", y_train.shape)
print("Testing Target:", y_test.shape)
```

```
Training Features: (2132, 71)
Testing Features: (534, 71)
Training Target: (2132,)
Testing Target: (534,)
```

```
# Step 13: Feature Scaling

scaler = StandardScaler()

X_train_scaled = pd.DataFrame(
    scaler.fit_transform(X_train),
    columns=X_train.columns,
    index=X_train.index
)

X_test_scaled = pd.DataFrame(
    scaler.transform(X_test),
    columns=X_test.columns,
    index=X_test.index
)

print("Feature scaling completed!")

print("\nScaled Training Data:")
display(X_train_scaled.head())

print("\nScaled Testing Data:")
display(X_test_scaled.head())
```

```
Feature scaling completed!

Scaled Training Data:
```

	Account length	Area code	Number vmail messages	Total day minutes	Total day calls	Total day charge	Total eve minutes	Total eve calls	Total eve charge	Total night minutes	...	State_TX	State_
1229	0.034820	-0.521107	-0.58321	1.700304	-1.022825	1.700625	-0.564073	1.648564	-0.564143	-0.627689	...	-0.143471	-0.1517
2001	1.430233	-0.686677	-0.58321	1.126024	-1.676615	1.125563	-0.935853	-0.112882	-0.934780	0.953599	...	-0.143471	-0.1517
1421	-0.548716	-0.521107	-0.58321	-0.661865	-0.519911	-0.661940	-1.272411	-0.465171	-1.273187	0.228346	...	6.970036	-0.1517
1346	0.795954	-0.686677	-0.58321	-0.180510	0.335045	-0.180899	0.167747	1.044639	0.167923	0.103508	...	-0.143471	-0.1517
86	-0.142778	-0.686677	-0.58321	1.086995	-1.777198	1.087299	-0.266649	-0.112882	-0.267172	-0.215524	...	-0.143471	-0.1517

5 rows × 71 columns

```
Scaled Testing Data:
```

	Account length	Area code	Number vmail messages	Total day minutes	Total day calls	Total day charge	Total eve minutes	Total eve calls	Total eve charge	Total night minutes	...	State_TX	State_
1396	1.151150	1.725917	-0.583210	0.180041	-1.173700	0.179881	-0.879107	1.396929	-0.879529	-0.294786	...	-0.143471	-0.1517
542	-0.751686	-0.521107	-0.583210	1.819248	0.184170	1.819792	-1.350681	1.044639	-1.351459	-1.465890	...	-0.143471	-0.1517
2275	-0.041294	1.725917	-0.583210	-0.953651	-1.123408	-0.953844	0.283195	0.994312	0.283028	-1.132988	...	-0.143471	-0.1517
1302	-0.827799	-0.521107	-0.583210	0.802643	-0.067287	0.803047	-0.489717	-0.767133	-0.490476	0.022264	...	-0.143471	-0.1517
560	1.709315	-0.521107	1.050356	-0.068999	-1.525740	-0.069385	0.212752	0.239407	0.211663	-0.522666	...	-0.143471	-0.1517

5 rows × 71 columns

```
# Step 14: Export Cleaned Dataset

output_file = "Telecom_Customer_Churn_Cleaned.csv"

df.to_csv(output_file, index=False)

print("Cleaned dataset saved successfully!")
print("File name:", output_file)
```

```
Cleaned dataset saved successfully!
File name: Telecom_Customer_Churn_Cleaned.csv
```

```
from google.colab import files
```

```
files.download("Telecom_Customer_Churn_Cleaned.csv")
```